Istanbul Technical University
Department of Artificial Intelligence and Data Engineering
YZV 322E — Applied Data Engineering · Spring 2026

FINAL PROJECT ANNOUNCEMENT

End-to-End Containerized Data Engineering Pipeline

Course	YZV 322E — Applied Data Engineering
Instructor	Dr. Mehmet Tunçel - tuncelm@itu.edu.tr
Teaching Assistants	Caner Özer, Abdullah Kavaklı, Emir Soydal - {ozerc, kavakli23, soydal25}@itu.edu.tr
Team Size	2 – 4 students
Weight	40% of the final course grade
Submission Deadline	**May 10, 23:59 (Türkiye time) — via Ninova**
Presentation Weeks	Week 13 and Week 14

1. Learning Outcomes

Upon successful completion of this project, students will be able to:

- Design a containerized (Docker / docker-compose based) data engineering architecture.
- Integrate multiple tools into an end-to-end, reproducible pipeline.
- Justify design decisions at every layer, from raw data ingestion to consumable output.
- Professionally present a team software product through a technical report, code repository, and live demo.
- Respond to technical questions from the evaluation committee in an analytical — rather than defensive — manner.

2. Teams

- Teams must consist of **2 to 4 students**. Individual submissions are not permitted.
- Each team designates a Team Lead, responsible for communication (not technical leadership).
- Work is expected to be distributed evenly. Each report must include an individual contribution table in its appendix.

3. Technical Requirements

3.1 Mandatory Docker Usage

The **entire project must run on Docker**. This includes not only databases and off-the-shelf services, but also all code written by the team (Python scripts, ETL jobs, API endpoints, helper scripts, etc.), which must run **inside their own containers**. Submissions that require Python, Java, or any other runtime to be installed directly on the evaluator's machine will **not be accepted**.

- All services must be defined under a single **docker-compose.yml** (or equivalent) file.
- Team-authored Python / application code must have its **own Dockerfile** and be brought up as a service from within the compose file. No **pip install** or **python script.py** commands should be required on the host machine.
- Persistent data must use named volumes; data must not be lost when a container is restarted.
- Inter-service communication must use the Docker network; no hard dependency on localhost or the host IP is allowed.

3.2 Required Use of Course Tools

In addition to Docker, the project must make meaningful, well-justified use of **at least two of the six course tools** listed below. "Meaningful use" means that the tool plays a critical role in the project's architecture; running a single trivial command does not qualify.

- PostgreSQL (relational storage)
- pgAdmin (database administration)
- Apache NiFi (visual data flow)
- Elasticsearch (document indexing and search)
- Kibana (dashboards and visualization)
- Apache Airflow (workflow orchestration)

In addition to these tools, teams are free to use any supplementary technology suited to their project (e.g., Apache Kafka, MinIO/S3, dbt, Spark, FastAPI, Grafana, Redis, Prometheus). Any additional tool must also run via Docker and must be briefly justified in the report.

3.3 Architectural Expectations

- The entire system must start with a single command: **docker compose up --build** should launch the pipeline end-to-end.
- Hardcoded credentials are prohibited; a .env file or an equivalent secret management mechanism must be used.
- After cloning the repository, the system must be up and running locally within 15 minutes at most.
- At least one end-to-end data flow must be automated (orchestrated).
- The system should recover reasonably from partial failures (e.g., a service being restarted).

3.4 Data Requirements

- The dataset may be open-source or synthetic; copyrighted data may not be used.

- Minimum scale: a single stream containing at least **10,000 records**, or a stream that merges multiple sources. The goal is to demonstrate that the pipeline actually flows — not to process massive data.
- The pipeline must run end-to-end **on a typical student laptop** (16 GB RAM, ~20 GB free disk). Teams working with larger datasets must prepare a sampled subset for the demo.
- When data privacy is a concern, appropriate anonymization techniques must be applied.

4. Deliverables

By the submission deadline, each team must submit the four components below together. Partial submissions receive partial credit; no component is optional.

4.0 Abstract

- Please submit the abstract one week before the final deliverables. (You can find the abstract submission separately on Ninova.) It must be in IEEE format and submitted as a PDF.

4.1 GitHub Repository (Mandatory)

- The repository link must be shared via Ninova, and access must be granted to the evaluation committee.
- A **README.md** must be present at the repository root and must include: project summary, architecture diagram, setup instructions, example commands, known limitations, and team members.
- The repository structure must be clear, organized into meaningful folders such as **/docker, /dags, /nifi, /sql, /src, /docs, /data (sample)**.
- Commit history must reflect individual team-member contributions. Bulk commits originating from a single contributor will be viewed unfavorably.
- A LICENSE file and .gitignore must be included. Large binary files must not be committed (a small sample dataset is sufficient).

4.2 Technical Report (PDF, 4–8 pages)

The report must contain the following sections. Academic formatting is expected: figures must be numbered, and references must follow IEEE or APA style.

1. Executive Summary — half a page maximum.
2. Problem statement and motivation.
3. Architectural design: component diagram, data flow diagram, and the rationale for design decisions.
4. Tools used and rationale for their selection (with comparison to alternatives).
5. Implementation details: data schema, DAG structure, NiFi flows, index design, etc.
6. Evaluation: performance measurements, data-quality checks, testing strategy.
7. Limitations and future work.
8. AI Usage Declaration (see Section 7).
9. Individual contribution table.
10. References.

This deliverable must be include both .tex and pdf file as zipped.

4.3 Presentation Slides (PDF + PPTX/Keynote)

- 8–12 slides, readable typography, minimum 20 pt font.
- Each slide should convey a single message; avoid wall-of-text slides.
- The architecture diagram and demo screenshots must appear in the slides.

4.4 Live Demo and Q&A Session

- Each team is allotted a total of **20 minutes**: 8 min presentation, 7 min live demo, 5 min Q&A.
- The demo must be run live on your local (laptop) environment; pre-recorded videos are not accepted (except in the case of unavoidable infrastructure failure).
- Every team member must be prepared to answer at least one technical question.

5. Evaluation and Grading (Rubric)

The rubric below shows the 100-point distribution for the final project. Each criterion is scored independently, and every level is defined in terms of observable behaviors. Final grades are assigned by the instructor and the teaching assistants.

Criterion	Weight	Component	Gold Standards
A. Live Demo and Q&A	**35 points**	Presentation weeks	The demo runs smoothly on the first try, with each team member confidently explaining their part. The team handles unexpected questions with clear, accurate answers and can make small live adjustments on the spot.
B. Technical Design and Architecture	**20 points**	Report + Code	The architecture is clearly structured with well-separated layers, and tool choices are justified through comparisons. Production concerns like failure handling, retries, and idempotency are also properly addressed.
C. Implementation and Code Quality	**20 points**	GitHub repository	The entire system is brought up with a single docker-compose up command, with all services and code running in containers and no host setup required. The codebase is modular and clean, with consistent naming, no linter issues, some automated testing, and a commit history shared across the team.
D. Report and Documentation	**15 points**	Report + README	The report is well-structured, with all sections complete, properly labeled figures, and consistent referencing. The README clearly enables someone unfamiliar with the repository to set up and run the system.
E. Presentation Slides	**5 points**	Slide deck	The slides are clear and focused, each conveying a single message, with visuals effectively supporting the architecture and results.
F. Transparency of AI Usage	**5 points**	AI Usage Declaration	The AI usage declaration clearly specifies the tools, purposes, and sections involved, explains how outputs were validated, and aligns with the individual contribution table.

Criterion	Weight	Component	Gold Standards
TOTAL	100 points		

6. Project Timeline

Week	Milestone	Expected Output
Final Submission	**May 10, 23:59**	All deliverables (report + slides + GitHub link) submitted via Ninova
Weeks 13 – 14	Presentations + demos + Q&A	20-minute live presentation

7. AI Usage Policy

This section extends the course's standing AI Tools Usage Policy and draws on international academic-integrity standards (UNESCO Guidance for Generative AI in Education and Research, 2023; Russell Group Principles on the Use of Generative AI Tools in Education, 2023; ICML/NeurIPS publication policies).

7.1 Permitted Uses

- Code completion, syntax reminders, and refactoring suggestions.
- Interpreting error messages and debugging hints.
- Conceptual clarifications (e.g., "what is Airflow XCom?").
- Language and style editing of the written report.
- Discussion of draft architectural ideas.

7.2 Prohibited Uses

- Having AI generate the **entire project or an identifiable large portion of it** and simply copy-pasting the output for submission.
- Filling the report with **unverified** AI output (fabricated citations, hallucinated metrics, nonexistent library functions).
- A team member showing zero familiarity with the system during Q&A — a typical indicator of an AI-generated project.
- Reformatting another team's project (or a template project from the internet) using AI.

7.3 Transparency Requirements

Every submission must include an AI Usage Declaration at the end of the report. This declaration must state the following three items:

11. The AI tool(s) used (including version information, if applicable).

12. The purpose of each use (e.g., "requested a JSON schema suggestion for the NiFi flow").
13. The team's validation method (e.g., "the suggested code was executed locally and unit tests passed").

If no AI tools were used, the declaration must still be included ("No AI tools were used in this project").

7.4 Responsibility

Students are fully responsible for the accuracy, functionality, and originality of everything they submit. Any errors arising from AI-generated content that was not validated by the team (hallucinated API calls, fabricated references, incorrect metrics, etc.) are attributed directly to the team and will reflect negatively in the evaluation.

7.5 Detection and Sanctions

- Probing questions may be asked during Q&A to assess the team's actual command of each component.
- When warranted, the team may be asked to make a small live change (e.g., add a column, run a DAG).
- Work judged to have been produced entirely by AI will receive a **grade of zero** and will be processed under the ITU Academic Integrity Policy.

8. Frequently Asked Questions

Can we use a tool not covered in the course (e.g., Kafka)?

Yes. As long as at least two of the six course tools are used, you may add any supplementary tool your project requires. Additional tools must also run via Docker and must be briefly justified in the report.

Can I run my Python code directly on my laptop?

No. All code in the project (Python scripts, ETL jobs, API endpoints, etc.) must run inside its own container. Submissions that require Python installation or pip install on the host machine are not accepted. For this, write a separate Dockerfile for the team's code and define it as a service in docker-compose.

Can we use cloud services (AWS, GCP, Azure)?

Yes, but you must also provide a local Docker Compose alternative so that the evaluation committee can run the system locally.

Can our team have 5 members?

No. Team size is capped at 4.

Can we submit the same project to another course?

Not without explicit written permission. Multiple submission constitutes academic dishonesty.

What happens if the internet goes down during the demo?

You must have a pre-recorded video backup ready for infrastructure failures. However, the video backup is played only in case of an actual failure; the default demo is live.

9. Closing Remarks

The final project is an opportunity to turn the content of this course into a working system built by a team rather than individuals. The goal is not perfection; it is producing a solution you can defend, whose decisions you can justify, and which you built together. The Ninova forum and office hours remain open for any questions you may encounter.

We wish you the best.

Dr. Mehmet Tunçel
YZV 322E — Applied Data Engineering
Spring 2026